Bahasa Pemrograman Tingkat Rendah (MWCA010)

Program Studi Teknik Informatika ▪ Fakultas Teknologi Informasi ▪ Universitas Bale Bandung

Direktif Assembler dan Alur Translasi Program

Pertemuan 7

Dosen Pengampu

Cecep Suwanda, S.Si., M.Kom.

Semester Ganjil

Capaian dan Tujuan Pembelajaran

Sub-CPMK 2.3

Menggunakan direktif assembler (`DB`, `DW`, `DD`, `EQU`, `ORG`, `SEGMENT`) dan menjelaskan alur translasi *source* → *object* → *executable* oleh assembler dan linker.

CPMK yang didukung: CPMK02 ▪ **CPL yang didukung: CPL02**

Setelah pertemuan ini, mahasiswa mampu:

1. membedakan instruksi mesin dan direktif assembler serta menjelaskan mengapa direktif tidak menghasilkan kode mesin;
2. menggunakan direktif data `DB`, `DW`, `DD`, dan `DT` beserta literal string, operator `DUP`, penanda `?`, dan pengaruh urutan *little-endian*;
3. membedakan direktif `EQU` dan `=` beserta akibatnya bila simbol didefinisikan ulang;
4. menggunakan direktif lokasi `ORG`, `SEGMENT`/`ENDS`, `ASSUME`, dan `END` sesuai bentuk program `.COM` maupun `.EXE`, serta memilih model memori `.MODEL` yang tepat;
5. menjelaskan tugas assembler (tabel simbol, *offset*) dan linker (*relocation*, resolusi simbol), lalu menelusuri alur `.asm` → `.obj` → `.exe` dengan TASM dan TLINK.

Peta Materi

1. **Instruksi Mesin, Direktif Assembler, dan Direktif Data**
   dua jenis baris kode sumber; `DB`, `DW`, `DD`, `DT`; `DUP`; penanda `?`; urutan *little-endian*
2. **Direktif Simbol, Lokasi, dan Struktur Program**
   `EQU` dan `=`; `ORG`, `SEGMENT`/`ENDS`, `ASSUME`, `END`; `.MODEL`; `PROC`/`ENDP`
3. **Peran Assembler dan Linker dalam Alur Translasi**
   dua lintasan assembler; tabel simbol; berkas `.lst`; resolusi simbol dan *relocation*; berkas `.map`; kesalahan perakitan

Praktik pada pertemuan ini

Merakit `direktif-data.asm` dengan TASM dan TLINK, lalu memeriksa berkas `.lst` dan `.map`.

BAGIAN

Instruksi Mesin, Direktif Assembler, dan Direktif Data

Dua Jenis Baris dalam Kode Sumber

Instruksi mesin — mnemonic seperti `MOV`, `ADD`, dan `INT` diterjemahkan menjadi **opcode**, yaitu kode mesin yang benar-benar dieksekusi prosesor. Instruksi hanya bekerja pada saat program berjalan (*run time*).

Direktif assembler — hanya berlaku pada saat perakitan (*assemble time*); ia memberi tahu assembler bagaimana menata program. Disebut *pseudo-op* karena bentuknya mirip instruksi tetapi **Penting:** tidak menghasilkan kode mesin apa pun.

Pada berkas `.lst`, baris instruksi memiliki kolom kode mesin, sedangkan baris direktif kosong:

```
 4                              .DATA
 5  0000  48 61 6C 6F        pesan   DB 'Halo'
 6                             panjang EQU 5
 8                              .CODE
 9  0000  B8 ----            START:  MOV AX, @DATA
10  0003  8E D8                      MOV DS, AX
```

Sumber: Bab IX, Dua Jenis Baris dalam Kode Sumber

Direktif Data dan Ukuran yang Disediakannya

Direktif	Fungsi	Contoh	Ukuran / efek
`DB`	Data 1 byte	`DB 0Ah`	1 byte per nilai
`DW`	Data 2 byte	`DW 1234h`	2 byte; *little-endian*
`DD`	Data 4 byte	`DD 12345678h`	4 byte; *little-endian*
`DT`	Data 10 byte	`DT 1.0`	10 byte (bilangan pecahan)
`DUP`	Mengulang satu pola	`DB 10 DUP(0)`	10 byte bernilai nol
`?`	Ruang tanpa nilai awal	`DB 8 DUP(?)`	8 byte tanpa isi
`$`	Penghitung lokasi	`EQU $ - pesan`	Nilai *offset* saat itu

Direktif yang mengisi memori (`DB`, `DW`, `DD`) tetap mempengaruhi isi berkas `.exe`. Yang tidak dihasilkannya adalah *instruksi*.

Sumber: Bab IX, Tabel ringkasan direktif assembler

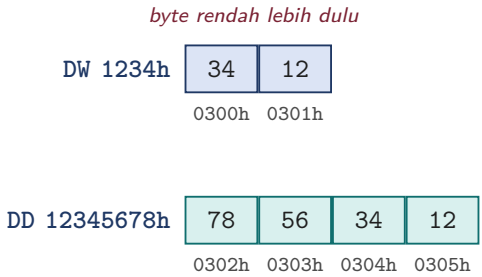
Bentuk Penulisan yang Sering Dipakai

1. **Literal string.** `DB 'Halo'` menyediakan empat byte: `48h`, `61h`, `6Ch`, `6Fh`. String tidak otomatis diakhiri apa pun, sehingga penanda akhir harus ditambahkan sendiri: `DB 'Halo', '$'`.
2. **Beberapa nilai sekaligus.** `DW 1, 2,` 3 menyediakan tiga kata — enam byte — dengan urutan sesuai penulisan.
3. **Operator `DUP`.** Bentuk `<jumlah> DUP(<nilai>)` mengulang satu pola; bentuk bersarang juga diizinkan, misalnya `DB 2 DUP(3 DUP(0))`.
4. **Penanda `?`.** Tanda tanya berarti sediakan ruang *tanpa* nilai awal, seperti pada `buffer DB 8 DUP(?)`.
5. **Penghitung lokasi `$`.** Simbol `$` berisi *offset* baris yang sedang dirakit, sehingga `panjang EQU $ - pesan` menghitung panjang string secara otomatis.

Sumber: Bab IX, Direktif Data

Urutan *Little-Endian* pada `DW` dan `DD`

Prosesor 8086 menyimpan bilangan multi-byte dengan urutan **little-endian**: **Penting:** byte berpangkat rendah disimpan pada alamat yang lebih kecil.



`nilai DW 1234h` *tidak* tersimpan sebagai `12h` lalu `34h`, melainkan `34h` lalu `12h`; aturan yang sama berlaku untuk `DD`. Urutan ini baru terlihat bila memori dibaca *byte per byte*.

Sumber: Bab IX, Urutan Little-Endian pada DW dan DD

Rangkuman Bagian 1

- Kode sumber memuat dua jenis baris: instruksi mesin yang menghasilkan opcode dan dieksekusi saat *run time*, serta direktif assembler (*pseudo-op*) yang hanya berlaku saat *assemble time* dan tidak menghasilkan kode mesin.
- `DB` menyediakan 1 byte, `DW` 2 byte, `DD` 4 byte, dan `DT` 10 byte. Operator `DUP` mengulang satu pola, sedangkan penanda `?` hanya menyediakan ruang tanpa nilai awal.
- Literal string tidak otomatis diakhiri; penanda `'$'` ditambahkan sendiri untuk layanan DOS fungsi `09h`. Simbol `$` sekaligus dapat dipakai menghitung *offset*.
- Bilangan multi-byte disimpan *little-endian*: byte berpangkat rendah berada di alamat lebih kecil, sehingga `DW 1234h` tersimpan `34h` lalu `12h`.

BAGIAN

Direktif Simbol, Lokasi, dan Struktur Program

Konstanta Simbolik: `EQU` dan =

`EQU` memberi nama pada sebuah nilai. Setelah `KELUAR EQU 4Ch` ditulis, setiap kemunculan `KELUAR` digantikan `4Ch` saat perakitan; nama semacam ini disebut **konstanta simbolik**. Direktif = berbentuk serupa, yaitu `URUTAN = 10`, tetapi sifatnya berbeda.

Aspek	`EQU`	=
Sifat simbol	Konstanta tetap sepanjang berkas	Nilai yang boleh berubah
Dapat didefinisikan ulang	Tidak; assembler melaporkan `Symbol already defined`	Ya; definisi terakhir yang berlaku
Jenis nilai yang boleh	Bilangan, alamat, teks, atau ekspresi yang dapat dihitung assembler	Wajib berupa bilangan
Pemakaian khas	Nama nomor fungsi DOS, ukuran tabel	Penghitung di dalam makro atau blok yang diulang

Sumber: Bab IX, Konstanta Simbolik

Definisi Ulang EQU Dibandingkan =

Definisi ulang EQU menimbulkan kesalahan perakitan:

```
LEBAR   EQU 80          ; definisi pertama
LEBAR   EQU 40          ; **Error** Symbol already defined
```

Pesan yang dikeluarkan TASM:

```
**Error** equ-salah.asm(2) Symbol already defined: LEBAR
**Error** equ-salah.asm(2) Cannot define symbol
2 errors
```

Dengan =, definisi ulang menjadi sah dan nilai terakhir yang berlaku:

```
URUTAN  = 10            ; nilai pertama
URUTAN  = 20            ; sah: nilai sekarang menjadi 20
```

Sumber: Bab IX, Definisi Ulang EQU

Direktif Lokasi dan Struktur Program

`ORG` Menetapkan penghitung lokasi (*location counter*). Pada program `.COM`, `ORG 100h` wajib ditulis karena DOS memuat program pada *offset* `100h`.

`SEGMENT` dan `ENDS` Menandai awal dan akhir sebuah segmen bernama. Setiap `SEGMENT` harus ditutup `ENDS` dengan nama segmen yang sama.

`ASSUME` Memberi tahu assembler register segmen mana yang dianggap menunjuk ke segmen tertentu, sehingga segmen *default* sebuah label dapat ditentukan. Direktif ini tidak mengubah isi register saat program berjalan.

`END` Menutup berkas sumber; baris sesudahnya diabaikan. Dengan label, `END START` sekaligus mencatat titik masuk (*entry point*). `END` tanpa label lolos dari assembler, tetapi ditolak linker.

`PROC` dan `ENDP` Menandai batas sebuah prosedur sehingga assembler dapat menentukan jenis pemanggilan. Keduanya tidak menghasilkan instruksi.

Sumber: Bab IX, Direktif Lokasi dan Struktur

Model Memori .MODEL

Model	Batas kode	Batas data	Jumlah segmen
TINY	64 KB	64 KB	Satu segmen untuk kode, data, dan *stack*; dipakai untuk .COM
SMALL	64 KB	64 KB	Satu segmen kode dan satu segmen data; pilihan lazim untuk pembelajaran
MEDIUM	Tidak dibatasi	64 KB	Satu segmen data, banyak segmen kode
COMPACT	64 KB	Tidak dibatasi	Satu segmen kode, banyak segmen data
LARGE	Tidak dibatasi	Tidak dibatasi	Banyak segmen; satu *array* maksimum 64 KB
HUGE	Tidak dibatasi	Tidak dibatasi	Seperti LARGE, tetapi *array* boleh melewati 64 KB

.MODEL juga menetapkan simbol bawaan seperti @DATA. Semakin besar model, semakin banyak segmen yang harus diurus program.

Sumber: Bab IX, Model Memori dan Struktur Prosedur

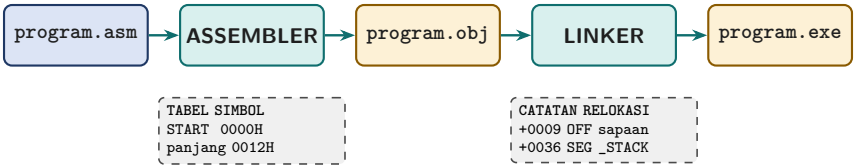
Rangkuman Bagian 2

- `EQU` menghasilkan konstanta tetap yang tidak dapat didefinisikan ulang, sedangkan `=` menghasilkan nilai yang boleh diganti berkali-kali — yang berlaku adalah definisi terakhir.
- Definisi ulang `EQU` dilaporkan sebagai `Symbol already defined`; gunakan `=` bila memang membutuhkan nilai yang berubah.
- `ORG` menetapkan penghitung lokasi (`ORG 100h` wajib pada `.COM`); `SEGMENT`/`ENDS` menandai batas segmen; `ASSUME` menetapkan segmen *default* bagi assembler; `END` menutup berkas dan dapat menetapkan titik masuk.
- `.MODEL` memilih model memori: `TINY` satu segmen untuk `.COM`, `SMALL` satu segmen kode dan satu segmen data. Model yang lebih besar memakai banyak segmen.
- `PROC` dan `ENDP` hanya menandai batas prosedur dan menentukan jenis pemanggilan, tanpa menghasilkan instruksi.

BAGIAN

Peran Assembler dan Linker dalam Alur Translasi

Alur Translasi: `.asm` → `.obj` → `.exe`



Assembler membangun tabel simbol dan menghasilkan kode mesin; linker menyelesaikan *relocation* dan menentukan alamat akhir.

Sumber: Bab IX, Alur translasi .asm .obj .exe

Tugas Assembler: Dua Lintasan

Assembler bekerja dalam dua lintasan (*two-pass*):

1. **Lintasan pertama.** Assembler membaca seluruh baris, memeriksa sintaks, menghitung besar setiap direktif data, dan mencatat setiap label beserta *offset*-nya ke dalam **tabel simbol**. Pada lintasan ini belum ada kode mesin yang ditulis ke berkas.

2. **Lintasan kedua.** Assembler menerjemahkan setiap instruksi menjadi opcode, menggabungkannya dengan *offset* dari tabel simbol, lalu menuliskan hasilnya ke berkas objek. Karena itu, label yang terletak *setelah* instruksi pemakainya tetap dapat dipakai.

Berkas `.lst`

Berkas `.lst` memperlihatkan hasil kedua lintasan sekaligus: kolom kode mesin, tabel simbol, dan *offset* yang sudah dihitung.

Sumber: Bab IX, Tugas Assembler

Membaca Tabel Simbol pada Berkas `.lst`

```
 4                                .DATA
 5  0000  48 61 6C 6F 2C 20 61 73 73 65  sapaan  DB 'Halo, ...'
 7  0012                          panjang EQU $ - sapaan
 8  0012  41 42 43 44             kata    DW 4241h, 4443h
 9  0016  78 56 34 12             nilai   DD 12345678h
10  001A  2A 2A 2A 2A             bintang DB 4 DUP('*')
11  001E  ??                      buffer  DB 8 DUP(?)
14  0000  B8 ----              START:  MOV AX, @DATA
17  0007  8D 16 0000 R                 LEA DX, sapaan
```

- Direktif data terlihat memakan ruang memori, sedangkan baris `.MODEL`, `.STACK`, dan `.CODE` tidak.
- `panjang` tercatat `12h`, yaitu 18, hasil perhitungan `$ - sapaan` oleh assembler.
- Tanda `R` berarti baris itu memuat **relokasi**: alamatnya masih menunggu linker.

Sumber: Bab IX, Cuplikan direktif-data.lst

Tugas Linker: Resolusi Simbol dan *Relocation*

Berkas `.obj` belum dapat dijalankan karena masih memuat **simbol eksternal**, yaitu nama yang belum punya alamat pasti. Linker melakukan tiga pekerjaan pokok:

1. **Resolusi simbol eksternal.** Setiap rujukan diberi alamat sesungguhnya. Contoh yang paling akrab adalah `@DATA`, yang baru memperoleh nomor segmen setelah linker menempatkan segmen data.

2. ***Relocation*.** Linker mencatat bagian kode mana saja yang harus disesuaikan bila program dimuat pada alamat dasar yang berbeda.

3. **Penentuan alamat akhir.** Linker menyusun seluruh segmen berurutan, menetapkan alamatnya, dan menggabungkan berkas objek dengan pustaka (*library*) bila diperlukan.

Ringkasan pekerjaan itu terekam pada berkas `.map` yang dibuat dengan opsi `/m`.

Sumber: Bab IX, Tugas Linker

Berkas .map: Alamat Akhir Setiap Segmen

```
Turbo Link  Version 5.0.  Copyright (c) 1992 Borland International

 Start  Stop   Length Name                    Class
 00000H 00035H 00036H _TEXT                  CODE
 00036H 0005BH 00026H _DATA                  DATA
 0005CH 0005BH 00000H _BSS                   BSS
 0005CH 00015BH 00100H _STACK                 STACK

Program entry point at 0000:0000
```

Segmen _DATA berukuran 26h byte, tepat sama dengan total direktif data pada berkas sumber. Titik masuk tercatat 0000:0000, yaitu label START.

Sumber: Bab IX, Cuplikan direktif-data.map

Alur Lengkap dari Baris Perintah DOS

```
C:\ASM>tasm /l direktif-data.asm
Turbo Assembler  Version 4.1
Assembling file:   direktif-data.asm
Error messages:    None
Passes:            2

C:\ASM>tlink /m direktif-data.obj
Turbo Link  Version 5.0.

C:\ASM>direktif-data
Halo, assembly!
ABCD****
```

Assembler menghasilkan `.obj` dan — karena opsi `/l` — `.lst`; linker menghasilkan `.exe` dan — karena opsi `/m` — `.map`. Keluaran `ABCD****` terjadi karena data dibaca *byte per byte* secara *little-endian*.

Sumber: Bab IX, Alur lengkap dari DOS

Empat Kesalahan Perakitan yang Paling Sering

1. **Simbol ganda.** Satu nama dipakai untuk dua label atau dua `EQU`. Pesan `Symbol already defined` hanya menyebut nama yang bentrok, sehingga letaknya harus dicari pada berkas `.lst`.
2. `EQU` **didefinisikan ulang.** Pesan `Symbol already defined` pada baris `EQU` kedua. Gunakan `=` bila memang membutuhkan nilai yang berubah.
3. `END` **tanpa label.** Perakitan selesai tanpa kesalahan, tetapi linker melaporkan tidak menemukan titik masuk program. Tuliskan `END START` sesuai nama label awal.
4. **Segmen tidak ditutup** `ENDS`**.** Muncul pesan `Block nesting error` atau `Missing ENDS`, dan semua baris sesudahnya ikut salah tafsir.

Sumber: Bab IX, Kesalahan perakitan yang umum

Rangkuman Bagian 3

- Assembler bekerja dua lintasan: lintasan pertama menghitung ukuran direktif dan membangun tabel simbol, lintasan kedua menghasilkan opcode dengan memakai *offset* dari tabel itu.
- Berkas `.lst` memperlihatkan hasil kedua lintasan, termasuk tanda `R` yang berarti baris tersebut masih menunggu *relocation* oleh linker.
- Linker menyelesaikan simbol eksternal (misalnya `@DATA`), melakukan *relocation*, dan menetapkan alamat akhir; ringkasannya terekam pada berkas `.map`.
- Alur lengkap: `tasm /l program.asm` → `tlink /m program.obj` → menjalankan `program.exe`.
- Empat kesalahan tersering: simbol ganda, `EQU` didefinisikan ulang, `END` tanpa label, dan segmen tanpa `ENDS`.

Contoh 1: Mendemonstrasikan Semua Direktif Data

```
.MODEL SMALL
.STACK 100h
.DATA
    sapaan  DB 'Halo, assembly!', 0Dh, 0Ah, '$'
    panjang EQU $ - sapaan       ; 18 byte
    kata    DW 4241h, 4443h
    nilai   DD 12345678h
    bintang DB 4 DUP('*')
    buffer  DB 8 DUP(?)
```

```
.CODE
START:
    MOV AX, @DATA
    MOV DS, AX
    MOV AH, 09h
    LEA DX, sapaan
    INT 21h
    MOV DL, BYTE PTR [kata]
    INT 21h
    MOV AH, 4Ch
    INT 21h
END START
```

Keluaran

```
Halo, assembly!
ABCD****
```

Data dibaca *byte per byte* dengan `BYTE PTR`; baris kedua keluaran membuktikan urutan *little-endian*.

Sumber: Bab IX, Contoh 1 — `code/asm/direktif-data.asm`

Contoh 2: Konstanta EQU

```
LAYAR       EQU 02h     ; cetak karakter
TULIS       EQU 09h     ; cetak string
KELUAR      EQU 4Ch     ; akhiri program
BARIS_BARU  EQU 0Dh     ; carriage return
AWAL_BARIS  EQU 0Ah     ; line feed

.DATA
  judul  DB 'Konstanta EQU', 0Dh, 0Ah, '$'
  isi    DB 'X = ', '$'
```

Keluaran

```
Konstanta EQU
X = 7
```

```
.CODE
START:
    MOV AX, @DATA
    MOV DS, AX
    MOV AH, TULIS         ; cetak judul
    LEA DX, judul
    INT 21h
    MOV AH, TULIS         ; cetak 'X = '
    LEA DX, isi
    INT 21h
    MOV AH, LAYAR         ; cetak '7'
    MOV DL, '7'
    INT 21h
    MOV AH, KELUAR
    INT 21h
END START
```

Sumber: Bab IX, Contoh 2 — `code/asm/equ-konstanta.asm`

Praktikum: Merakit dengan TASM dan TLINK

Judul praktikum

Merakit dengan TASM dan TLINK serta memeriksa berkas `.lst` dan `.map` ▪ **Sub-CPMK:** 2.3
▪ **Waktu:** 100 menit ▪ **Bentuk:** individu

Alat dan bahan

- DOSBox dengan direktori kerja ter-*mount*
- TASM dan TLINK versi DOS
- `direktif-data.asm` dan `equ-konstanta.asm`
- Lembar kerja pengamatan

Langkah kerja

1. `tasm /l direktif-data.asm`; catat lintasan dan kesalahan
2. `tlink /m direktif-data.obj`; jalankan `.exe`
3. Periksa `.lst`: baris direktif dan baris bertanda `R`
4. Periksa `.map`: hitung besar `_DATA` dan `_TEXT`
5. Sengaja buat kesalahan `EQU` dan `END` tanpa label

Dikumpulkan: berkas `.asm` beserta `.obj`, `.lst`, `.map`, dan `.exe`, serta catatan kesalahan yang disengaja dan cara memperbaikinya.

Latihan

1. Jelaskan perbedaan instruksi mesin dan direktif assembler, serta mengapa direktif disebut *pseudo-op*. Berikan dua contoh direktif yang mengisi memori dan dua contoh yang tidak mempengaruhi isi berkas `.exe`.

2. Sebutkan direktif data yang tepat, lalu tuliskan deklarasinya: (a) satu nilai `0Ah` untuk pindah baris; (b) tabel delapan nilai 16-bit `0000h`; (c) penampung 32 byte yang diisi saat program berjalan; (d) string `'SEMARANG'` yang siap dicetak fungsi `09h`.

3. Diberikan blok data `a DB 'TEST'`, `b DW 2 DUP(0)`, `c DD 1, 2`, `d DB 3 DUP(?)`, dan `e DT 0`. Hitung total byte yang disediakan dan tuliskan *offset* setiap label.

4. Tuliskan urutan byte di memori untuk `DW 0102h` dan `DD 03040506h`, lalu jelaskan mengapa urutannya demikian.

5. Sebuah berkas sumber berakhir dengan `END` tanpa nama label. Apakah TASM melaporkan kesalahan? Apakah TLINK melaporkan kesalahan? Bagaimana cara memperbaikinya?

Rangkuman

- Kode sumber assembly terdiri atas dua jenis baris: instruksi mesin yang menghasilkan opcode dan dieksekusi saat program berjalan, serta direktif assembler yang hanya berlaku saat perakitan dan tidak menghasilkan kode mesin.
- `DB` menyediakan satu byte, `DW` dua byte, `DD` empat byte, dan `DT` sepuluh byte. `DUP` mengulang satu pola, sedangkan `?` hanya menyediakan ruang tanpa nilai awal.
- Bilangan multi-byte disimpan *little-endian*, sehingga `DW 1234h` tersimpan sebagai byte `34h` lalu `12h`.
- `EQU` membuat konstanta tetap yang tidak dapat didefinisikan ulang, sedangkan `=` membuat nilai yang boleh diganti berkali-kali.
- `ORG`, `SEGMENT`/`ENDS`, `ASSUME`, dan `END` mengatur letak dan struktur program; `END` yang diberi label sekaligus menetapkan titik masuk.
- Assembler membangun tabel simbol dan menghitung *offset*; linker menyelesaikan simbol eksternal, melakukan *relocation*, dan merekam alamat akhir pada berkas `.map`.

Cek Pemahaman

1. Pernyataan yang benar tentang direktif assembler adalah ...

   1.1 direktif menghasilkan opcode seperti instruksi
   1.2 direktif hanya berlaku saat perakitan dan tidak menghasilkan kode mesin
   1.3 direktif dieksekusi prosesor saat program berjalan
   1.4 direktif tidak berpengaruh pada berkas `.exe`

2. Deklarasi `nilai DW 1234h` menempatkan byte di memori dengan urutan ...

   2.1 `12h` lalu `34h`
   2.2 `34h` lalu `12h`
   2.3 `00h` lalu `12h`
   2.4 `43h` lalu `21h`

3. Tugas yang dikerjakan linker, bukan assembler, adalah ...

   3.1 menghitung *offset* label dalam satu segmen
   3.2 menerjemahkan mnemonic menjadi opcode
   3.3 menyelesaikan simbol eksternal dan melakukan *relocation*
   3.4 membaca direktif `EQU`

Penutup

Sebelum pertemuan berikutnya

- Selesaikan Tugas 3: berkas hasil perakitan, cuplikan `.lst` dan `.map`, serta catatan kesalahan perakitan yang disengaja.
- Tuliskan deklarasi `DB`, `DW`, dan `DD` untuk tabel lima nilai, lalu hitung *offset* setiap label.
- Baca **Bab X** — Instruksi Transfer Data dan Aritmatika.

Terima kasih

Bahasa Pemrograman Tingkat Rendah (MWCA010) ▪ Semester Ganjil